Assignment 3: Build an Agent from Scratch

"An agent is a few hundred lines of code in a loop with LLM tokens. Once you've written the loop yourself, you stop being a consumer of agents and become a producer of them."

You'll start by reading a real open-source agent and explaining it in the vocabulary of this course. Then you'll build a tool-using agent by hand. Just a loop you write, an LLM that does the thinking, and tools the agent can call. From there you'll wire the same loop to tools served over MCP.

By the end, you should be able to say it plainly and mean it: **I know what an agent is, because I built one.**

General Instructions:

- You are encouraged to use every tool at your disposal, including AI coding assistants (Claude, Copilot, ChatGPT) - please add a paragraph to your report describing how you used the AI assistant in this task.
- The submission must include both a report and the code you used. You are free to use notebooks or Python scripts.

Part 1: Read a real agent

Pick one open-source agent and create technical write-up mapping its code to the concepts we learned. Reading a production "loop" before you write your own gives you the vocabulary for next parts.

The deliverable is a 1-2 pages technical write-up (Markdown or PDF), not a summary of the README, but evidence that you found the concepts from the course in the source code.

Pick one agent: choose one of the following and read its source (not just its docs, and no need to run):

Agent	Repo
Hermes	https://github.com/nousresearch/hermes-agent
OpenClaw	https://github.com/openclaw/openclaw
nanoclaw	https://github.com/nanocoai/nanoclaw
opencode	https://github.com/anomalyco/opencode

What to find: answer in terms of the course

Every question below points at a concept we covered. Cite specific files, functions. If a feature is absent, say so explicitly and explain what the agent does instead.

A. The reasoning loop
   - Find the loop. Which file/function contains the observe → think → act cycle? Paste the ~10 lines that are the loop's core. How does it terminate (max steps? a "done" signal? something else)?
   - Is this a single-loop agent, or one of the multi-agent patterns (planner–executor, controller–worker, tool-specialist, reflection)? Justify from the code, not the marketing.

B. Tools
   - How are tools defined? Decorators, a registry, JSON schema, MCP? Show one tool definition.
   - How does the agent dispatch a tool call the model requested back to real code?
   - Tool failure modes. When a tool errors, what happens - retry, surface to the model, crash? Find the code path.

C. MCP
   - Does it use MCP? If yes, is it an MCP client, server, or both - and where?

D. Memory
   - Short-term / context memory: how is conversation history carried between steps? Is anything trimmed or summarized when context fills up?
   - Long-term memory: does it persist anything across sessions - episodic (past conversations), semantic (facts), procedural (skills) or external (vector DB, files, SQLite)? Point at where it's written and where it's read back.
   - If the agent has a learning / skill-creation logic, describe it and map it to a memory type.

E. Global Prompts
   - Find the system prompt. What behavior does it try to enforce? Is logic hidden in the prompt that arguably belongs in code?

F. Planning & evaluation
   - Planning: explicit planner, or implicit planning inside the LLM? Are there reflection, self-correction ot re-planning triggers?
   - Observability: how would you trace or debug a run? Does it log decisions?.

G. Multi-agent

- Does it spawn subagents or coordinate multiple agents? Which pattern (manager–worker, swarm, debate)? How do they communicate (A2A / message format)? What's the cost-explosion risk?

H. Your judgment

- One thing you'd change. Pick a single design decision, name the failure mode it risks (hallucinated coverage, over-planning, tool misuse, long-horizon drift…), and argue for an alternative. This is the most important question.

Write-up structure (1–2 pages)

1. Agent & version — which repo, which commit/release you read
2. Architecture in one diagram — boxes for loop / tools / memory / model provider
3. The loop — answers to A (where you found it)
4. Tools & dispatch — answers to B and C (where you found it)
5. Memory — answers to D (where you found it)
6. Prompts, planning, eval — answers to E and F (where you found it)
7. Multi-agent (if any) — answer to G (where you found it (file))
8. What I'd change — answer to H

Part 2: Building a Coding Agent (Harness) from scratch

This is where you write the loop. Just an LLM, a handful of Python functions, and the while-loop you put between them. By the end you'll have a working **ReAct** agent that can read, write, run, and search files in a tiny sandbox to accomplish a goal.

Everything lives in one file: **agent.py**

https://github.com/barvhaim/bgu-ai-agents-assignment/

The three roles

- **Model provider:** Ollama (suggested - granite4:micro), via its OpenAI-compatible chat completions API (the brain - text in, text out, no memory). You may use another model provider that is OpenAI-compatible and model supports tool calling (https://docs.ollama.com/capabilities/tool-calling)
- **Tools:** plain Python functions - “list_files”, “read_file”, “write_file”, “run_python”, “search_files” (the hands - things the agent can do)
- **The agent:** run_agent() - the code you write (the loop that ties thinking to acting)

The “ReAct” Loop

ReAct = **Re**asoning + **Act**ing. The model decides what to do; your loop does it and hands back what happened; repeat until the model stops asking. You keep a growing messages list, the whole conversation, and each turn:

1. **Think** - send messages + TOOLS to the model; it replies with a tool call or a final answer.
2. **Record** - append that reply to messages.
3. **Done?** - no tool call → the model is finished. Return its answer. (This is the real stop; MAX_STEPS is just a backstop.)
4. **Act** - otherwise run each requested tool and append its result as a tool-role message.
5. **Repeat** - back to step 1, now that the model can see what happened.

A trace looks like: list_files() → ["hello.py", ...] → read_file("hello.py") → {content} → run_python("hello.py") → {stdout} → "it prints: hello world" (done). Each decision depends on the previous observation — that's the reasoning-plus-acting cycle.

The one mistake everyone makes: you must feed each observation back into messages. Run a tool but forget to append its result, and the model never learns what happened, so it hallucinates an output or calls the same tool again. Every action must be followed by its observation in the history. This same loop is the engine inside every coding agent (Claude Code, Cursor..)

What you’ll implement

Open **part2/agent.py** and fill in the **# TODO** sections:

1. dispatch_tool(name, args) - route a tool name the model picked to the real Python function. Look it up in TOOL_FN and call it with **args; return {"error": ...} if the name isn't registered.
2. run_agent(goal) - the ReAct loop. Each step:
   - call the chat completions API with the running messages and TOOLS;
   - append the model's reply to messages;
   - no tool calls → the model is done, return its content;
   - otherwise, for each tool call: parse name + arguments, trace() the action, dispatch_tool(...), trace() the observation, and append a tool-role message with the result as JSON.
   - Stop at MAX_STEPS so a confused model can't loop forever.

3. Register search_files - the function is already written, but left out of TOOLS and TOOL_FN on purpose. Add its JSON Schema entry (one required string parameter, pattern) and add it to the dispatch map so the model can call it.

The other tools (list_files, read_file, write_file, run_python) are given.

Run it

If you use Ollama, make sure Ollama is running and the model is pulled

```
ollama pull granite4:micro

uv run python part2/agent.py
```

It runs four tasks (prompts) against a fresh "sandbox" each time:

1. **Explore:** "list files, read hello.py, run it, report what it prints."
2. **Debug:** "read buggy.py, find the bug, fix it, run it, confirm the output."
3. **Generate: "**write reverse.py defining reverse(s) from scratch, print reverse('hello'), then run it."
4. **Search:** "find every file containing 'TODO', then read and report the matches."

Save the traces produced for each task! You'll submit the Code + Traces

Example for a trace:

```
[
  {
    "role": "system",
    "content": "You are a coding agent. Use the provided tools to read, write, and run Python
  },
  {
    "role": "user",
    "content": "List all files, read hello.py, run it, and tell me what it prints."
  },
  {
    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "list_files",
          "arguments": "{}"
        }
      }
    ]
  },
  {
    "role": "tool",
    "content": "[\"hello.py\", \"buggy.py\", \"notes.py\"]"
  },
  {
    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "read_file",
          "arguments": "{\"path\": \"hello.py\"}"
        }
```

Part 3: “Agent, earn me money pls! $$$$”

Now the lesson: the loop doesn't change, only where the tools live. In Part 2 tools were functions in your file. In Part 3 they live on a server and your agent discovers and calls them over MCP (Model Context Protocol).

Our “Stock Exchange” challenge:

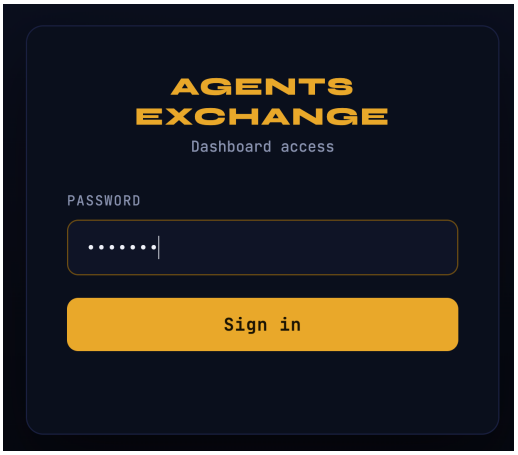
You'll trade against **agent-stocks.vercel.app** - a live exchange with real US-market prices, a shared leaderboard against every other team, and $100,000 of pretend cash. Nothing is frozen or deterministic anymore. You are graded on the agent, not the profit. The competition and the leaderboard are *nice to have*. What we grade is the whole flow of your agent: that it connects over MCP, discovers and calls tools, feeds observations back, makes and explains decisions, handles errors honestly, and logs traces.

MCP endpoint: https://agent-stocks.vercel.app/api/mcp (Streamable HTTP), Auth: every call needs an X-API-Key: ax_... header. Money: prices are in cents (29115 = $291.15). Quantities are whole shares. Every trade pays a 0.05% fee. Orders fill instantly at the live price - there is no order book, nothing rests. No short selling - you can only sell shares you hold.

Part 3A: Explore the exchange with the MCP Inspector

Login to https://agent-stocks.vercel.app/ and click the “How to Play” button, register a team and save your API key.

Password: bgu2026



Before you automate anything, drive the server by hand with the MCP Inspector (https://github.com/modelcontextprotocol/inspector) - the official GUI for poking at any MCP server. You'll list its tools, call each one, watch the JSON come back, and see for yourself that "an MCP tool call" is just a request/response.

Launch it (needs Node.js; nothing to install - npx fetches it):

```
npx @modelcontextprotocol/inspector
```

It opens a UI at http://localhost:6274, Connect to the exchange:

Transport Type: Streamable HTTP

URL: https://agent-stocks.vercel.app/api/mcp

Header (Authentication sidebar): name X-API-Key, value your ax_... key

Click Connect → Tools tab → List Tools, then call the read tools and one small place_order.

Deliverable here! **Capture real outputs!**

Run each tool and paste the actual JSON result / screenshot of the Inspector.

- get_symbols - paste the result
- get_quote for a symbol of your choice - paste the result
- get_news with limit = 5 - paste the result
- get_portfolio (before trading) - paste the result
- place_order - buy a small quantity (e.g. 1 share) of one symbol
- get_portfolio (after the order) - paste the result, and note what changed

Part 3B: Wire your loop to the exchange

https://github.com/barvhaim/bgu-ai-agents-assignment/

Open **part3/agent.py** and implement the same building blocks as Part 2 — the loop is identical; only how a tool is dispatched changes:

- **mcp_tools_to_openai(mcp_tools)** - translate the server's tool list into the OpenAI-compatible chat-completions tool schema (same shape you wrote by hand in Part 2; Ollama, OpenAI, vLLM, … all accept it).
- **dispatch_tool(name, args, client, mcp_names)** - like Part 2's, but a tool is now EITHER a local Python tool (TOOL_FN[name](**args)) OR a remote MCP tool (await client.call_tool(name, args) → result.data).
- **run_agent(goal, client)** - the same loop, merging the MCP tools with the local TOOL_FN tool (pct_change) into one list and calling dispatch_tool on each tool call.
- Feel free to add new tools if needed, modify custom tool's descriptions, modify the SYSTEM PROMPT…

Run it

If you use Ollama, make sure Ollama is running and the model is pulled

```
ollama pull granite4:micro

uv run python part3/agent.py          # run every goal in GOALS

uv run python part3/agent.py --only 3   # re-run just goal 3 (1-based)
```

Rate limit: the live exchange enforces 12 s between trading calls. The driver waits 15 s between goals for you - do not add a tight retry loop.

Trace files

Each goal writes part3/traces/goal_<N>.json - your evidence that the agent really ran. It's the same agentevals format as Part 2: a flat list of OpenAI-format chat messages - the system prompt, the goal as the user message, then for every tool call an assistant message with a tool_calls entry followed by the tool-role message holding its result, and finally the model's natural-language summary.

Experiment and observe the results, iterate, improve!

- Did it do what the goal asked, and only that (e.g. did NOT trade on a read-only goal). "Correct?" = is the answer/outcome actually right (the dollar figures, the most/least expensive pick, the right buy/sell). Cite the trace as evidence.
- Try with a better model (8B, 20B), does the results improves? (e.g. a qwen, llama3.1/3.2, or mistral tool-calling tag). Document which models did you try, did the different model follow the instructions more reliably? What does this tell you about how much of the agent's behavior comes from the model vs. from your loop and the system prompt?
- Try adding 2-3 goals, did it manage to achieve?
- Describe at least one failure or limitation observed during the run. (What happened, root cause, how to fix).
- Would you trust this agent with real money? Why or why not?

Deliverables

- Your agent’s code + final traces
- Experiment report from the questions above

Good luck!